

class08

Tanieng Huang

Class08 mini project

Preparing the data

```
# Read the CSV file; row.names=1 sets column 1 (id) as row names
fna.data <- "WisconsinCancer.csv"
wisc.df <- read.csv(fna.data, row.names=1)

# Preview the first 4 rows
head(wisc.df, 4)
```

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean
842302	M	17.99	10.38	122.80	1001.0
842517	M	20.57	17.77	132.90	1326.0
84300903	M	19.69	21.25	130.00	1203.0
84348301	M	11.42	20.38	77.58	386.1

	smoothness_mean	compactness_mean	concavity_mean	concave.points_mean
842302	0.11840	0.27760	0.3001	0.14710
842517	0.08474	0.07864	0.0869	0.07017
84300903	0.10960	0.15990	0.1974	0.12790
84348301	0.14250	0.28390	0.2414	0.10520

	symmetry_mean	fractal_dimension_mean	radius_se	texture_se	perimeter_se
842302	0.2419	0.07871	1.0950	0.9053	8.589
842517	0.1812	0.05667	0.5435	0.7339	3.398
84300903	0.2069	0.05999	0.7456	0.7869	4.585
84348301	0.2597	0.09744	0.4956	1.1560	3.445

	area_se	smoothness_se	compactness_se	concavity_se	concave.points_se
842302	153.40	0.006399	0.04904	0.05373	0.01587
842517	74.08	0.005225	0.01308	0.01860	0.01340
84300903	94.03	0.006150	0.04006	0.03832	0.02058

84348301	27.23	0.009110	0.07458	0.05661	0.01867
	symmetry_se	fractal_dimension_se	radius_worst	texture_worst	
842302	0.03003		0.006193	25.38	17.33
842517	0.01389		0.003532	24.99	23.41
84300903	0.02250		0.004571	23.57	25.53
84348301	0.05963		0.009208	14.91	26.50
	perimeter_worst	area_worst	smoothness_worst	compactness_worst	
842302	184.60	2019.0		0.1622	0.6656
842517	158.80	1956.0		0.1238	0.1866
84300903	152.50	1709.0		0.1444	0.4245
84348301	98.87	567.7		0.2098	0.8663
	concavity_worst	concave.points_worst	symmetry_worst		
842302	0.7119		0.2654	0.4601	
842517	0.2416		0.1860	0.2750	
84300903	0.4504		0.2430	0.3613	
84348301	0.6869		0.2575	0.6638	
	fractal_dimension_worst				
842302		0.11890			
842517		0.08902			
84300903		0.08758			
84348301		0.17300			

Create a new data.frame that omits this first column:

```
# Remove the diagnosis column (column 1) to get only numeric features
wisc.data <- wisc.df[,-1]

# Save diagnosis as a factor vector for later evaluation
diagnosis <- as.factor(wisc.df$diagnosis)
```

- **Q1.** How many observations are in this dataset?

```
nrow(wisc.data)
```

```
[1] 569
```

There are 569 observations in this dataset.

- **Q2.** How many of the observations have a malignant diagnosis?

```
sum(diagnosis == "M")
```

```
[1] 212
```

212 of the observations have a malignant (M) diagnosis.

- **Q3.** How many variables/features in the data are suffixed with `_mean`?

```
# Use grep() to find column names ending in "_mean"
length(grep("_mean", colnames(wisc.data)))
```

```
[1] 10
```

```
# To see which ones:
grep("_mean", colnames(wisc.data), value=TRUE)
```

```
[1] "radius_mean"          "texture_mean"        "perimeter_mean"
[4] "area_mean"           "smoothness_mean"     "compactness_mean"
[7] "concavity_mean"      "concave.points_mean" "symmetry_mean"
[10] "fractal_dimension_mean"
```

10 variables are suffixed with `_mean`.

3. Principal Component Analysis

Performing PCA

Check the mean and standard deviation of the features

```
# Check column means and standard deviations
colMeans(wisc.data)
```

radius_mean	texture_mean	perimeter_mean
1.412729e+01	1.928965e+01	9.196903e+01
area_mean	smoothness_mean	compactness_mean
6.548891e+02	9.636028e-02	1.043410e-01
concavity_mean	concave.points_mean	symmetry_mean
8.879932e-02	4.891915e-02	1.811619e-01
fractal_dimension_mean	radius_se	texture_se
6.279761e-02	4.051721e-01	1.216853e+00

perimeter_se	area_se	smoothness_se
2.866059e+00	4.033708e+01	7.040979e-03
compactness_se	concavity_se	concave.points_se
2.547814e-02	3.189372e-02	1.179614e-02
symmetry_se	fractal_dimension_se	radius_worst
2.054230e-02	3.794904e-03	1.626919e+01
texture_worst	perimeter_worst	area_worst
2.567722e+01	1.072612e+02	8.805831e+02
smoothness_worst	compactness_worst	concavity_worst
1.323686e-01	2.542650e-01	2.721885e-01
concave.points_worst	symmetry_worst	fractal_dimension_worst
1.146062e-01	2.900756e-01	8.394582e-02

```
apply(wisc.data,2,sd)
```

radius_mean	texture_mean	perimeter_mean
3.524049e+00	4.301036e+00	2.429898e+01
area_mean	smoothness_mean	compactness_mean
3.519141e+02	1.406413e-02	5.281276e-02
concavity_mean	concave.points_mean	symmetry_mean
7.971981e-02	3.880284e-02	2.741428e-02
fractal_dimension_mean	radius_se	texture_se
7.060363e-03	2.773127e-01	5.516484e-01
perimeter_se	area_se	smoothness_se
2.021855e+00	4.549101e+01	3.002518e-03
compactness_se	concavity_se	concave.points_se
1.790818e-02	3.018606e-02	6.170285e-03
symmetry_se	fractal_dimension_se	radius_worst
8.266372e-03	2.646071e-03	4.833242e+00
texture_worst	perimeter_worst	area_worst
6.146258e+00	3.360254e+01	5.693570e+02
smoothness_worst	compactness_worst	concavity_worst
2.283243e-02	1.573365e-01	2.086243e-01
concave.points_worst	symmetry_worst	fractal_dimension_worst
6.573234e-02	6.186747e-02	1.806127e-02

```
# Perform PCA with scaling (scale.=TRUE)
wisc.pr <- prcomp(wisc.data, scale.=TRUE, center=T)

# View summary - shows proportion of variance for each PC
summary(wisc.pr)
```

Importance of components:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7
Standard deviation	3.6444	2.3857	1.67867	1.40735	1.28403	1.09880	0.82172
Proportion of Variance	0.4427	0.1897	0.09393	0.06602	0.05496	0.04025	0.02251
Cumulative Proportion	0.4427	0.6324	0.72636	0.79239	0.84734	0.88759	0.91010
	PC8	PC9	PC10	PC11	PC12	PC13	PC14
Standard deviation	0.69037	0.6457	0.59219	0.5421	0.51104	0.49128	0.39624
Proportion of Variance	0.01589	0.0139	0.01169	0.0098	0.00871	0.00805	0.00523
Cumulative Proportion	0.92598	0.9399	0.95157	0.9614	0.97007	0.97812	0.98335
	PC15	PC16	PC17	PC18	PC19	PC20	PC21
Standard deviation	0.30681	0.28260	0.24372	0.22939	0.22244	0.17652	0.1731
Proportion of Variance	0.00314	0.00266	0.00198	0.00175	0.00165	0.00104	0.0010
Cumulative Proportion	0.98649	0.98915	0.99113	0.99288	0.99453	0.99557	0.9966
	PC22	PC23	PC24	PC25	PC26	PC27	PC28
Standard deviation	0.16565	0.15602	0.1344	0.12442	0.09043	0.08307	0.03987
Proportion of Variance	0.00091	0.00081	0.0006	0.00052	0.00027	0.00023	0.00005
Cumulative Proportion	0.99749	0.99830	0.9989	0.99942	0.99969	0.99992	0.99997
	PC29	PC30					
Standard deviation	0.02736	0.01153					
Proportion of Variance	0.00002	0.00000					
Cumulative Proportion	1.00000	1.00000					

- **Q4.** From your results, what proportion of the original variance is captured by the first principal component (PC1)?

The proportion of original variance captured by PC1 is 44.27% (0.4427).

- **Q5.** How many principal components (PCs) are required to describe at least 70% of the original variance in the data?

Look at the Cumulative Proportion row: $PC1 = 0.4427$, $PC1+PC2 = 0.6324$, $PC1+PC2+PC3 = 0.7264$. So 3 PCs are required for at least 70%.

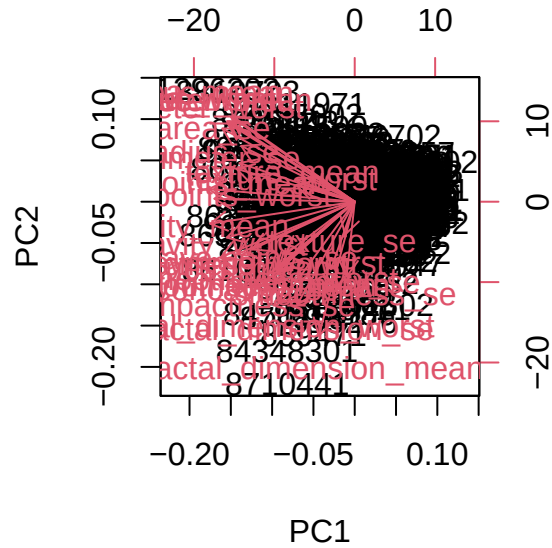
- **Q6.** How many principal components (PCs) are required to describe at least 90% of the original variance in the data?

The cumulative proportion reaches 0.91010 at PC7. So 7 PCs are required for at least 90%.

Interpreting PCA results

- **Q7.** What stands out to you about this plot? Is it easy or difficult to understand? Why?

```
# Q7: Create a biplot
biplot(wisc.pr)
```

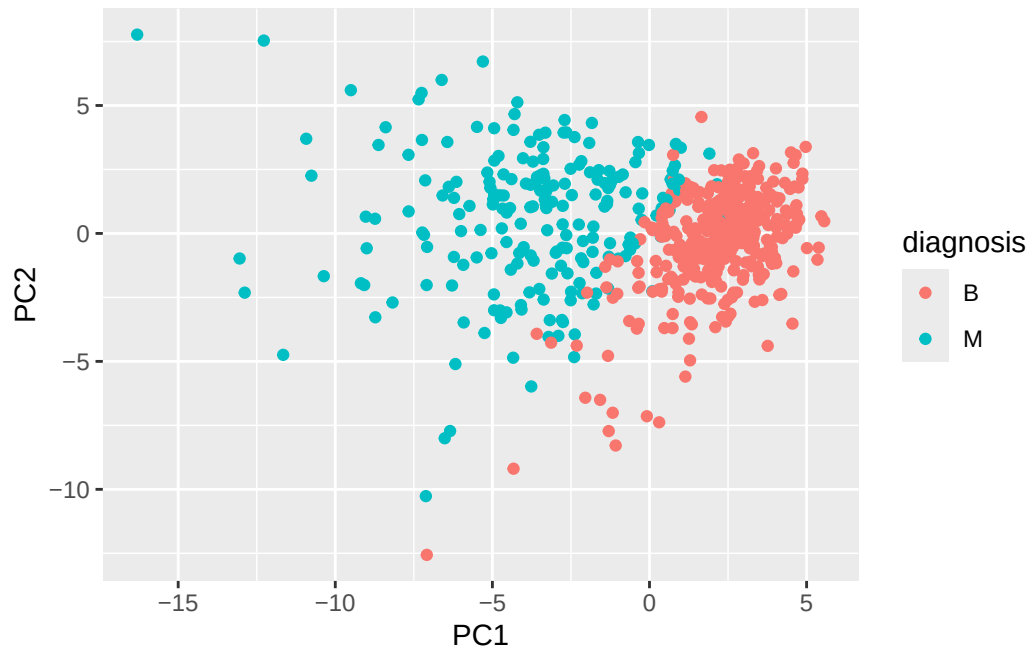


This biplot is very difficult to understand. The row names which are the sample IDs are used as plotting characters to create a overlapping mess of text. With 569 observations and 30 variables, the plot is unreadable. We need a simpler scatter plot instead.

PC1 vs PC2 scatter plot colored by diagnosis:

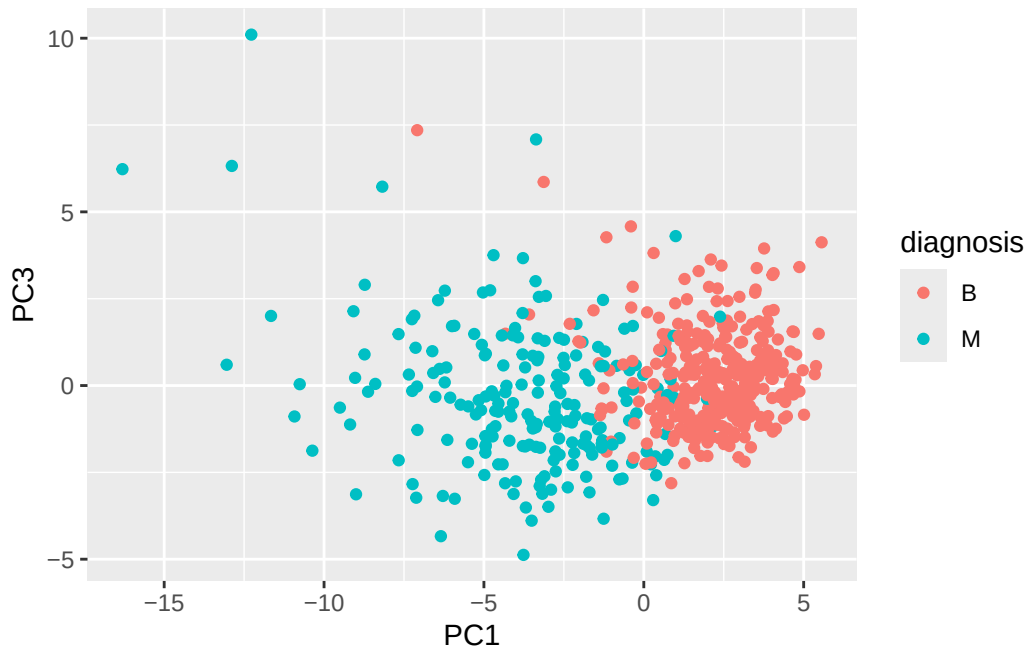
```
# Scatter plot observations by components 1 and 2
# colored by diagnosis
library(ggplot2)

ggplot(wisc.pr$x) +
  aes(PC1, PC2, col=diagnosis) +
  geom_point()
```



- **Q8.** Generate a similar plot for principal components 1 and 3. What do you notice about these plots?

```
# Repeat for components 1 and 3
# PC1 vs PC3
ggplot(wisc.pr$x) +
  aes(PC1, PC3, col=diagnosis) +
  geom_point()
```



The PC1 vs PC3 plot still shows separation along PC1 (left = M, right = B), but there is more overlap between the two groups compared to the PC1 vs PC2 plot. This makes sense because PC2 captures more variance than PC3, so it provides a better secondary axis for distinguishing the groups.

Variance explained

Scree Plot

Calculate the variance of each principal component by squaring the `sdev` component of `wisc.pr` (i.e. `wisc.pr$sdev^2`). Save the result as an object called `pr.var`.

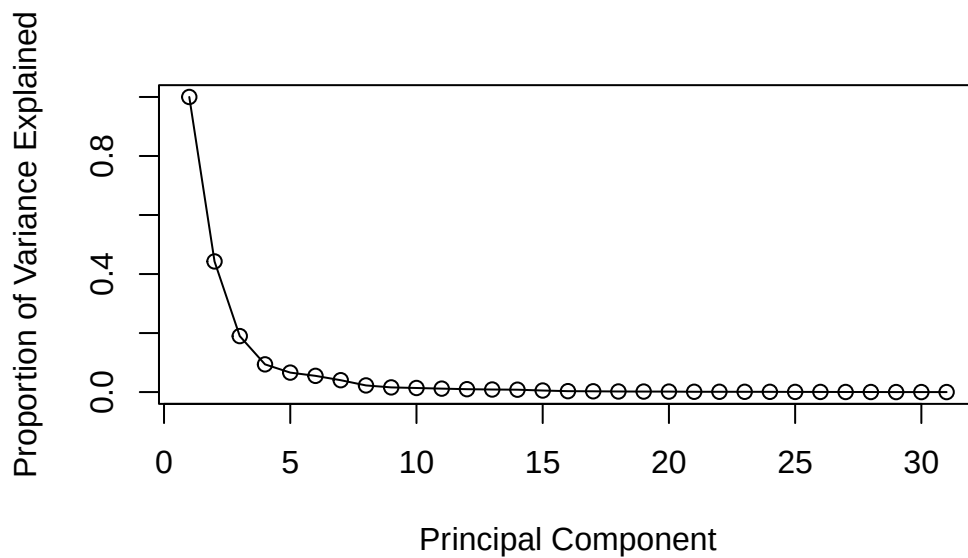
```
# Calculate variance of each PC (square the standard deviations)
pr.var <- wisc.pr$sdev^2
head(pr.var)
```

```
[1] 13.281608  5.691355  2.817949  1.980640  1.648731  1.207357
```

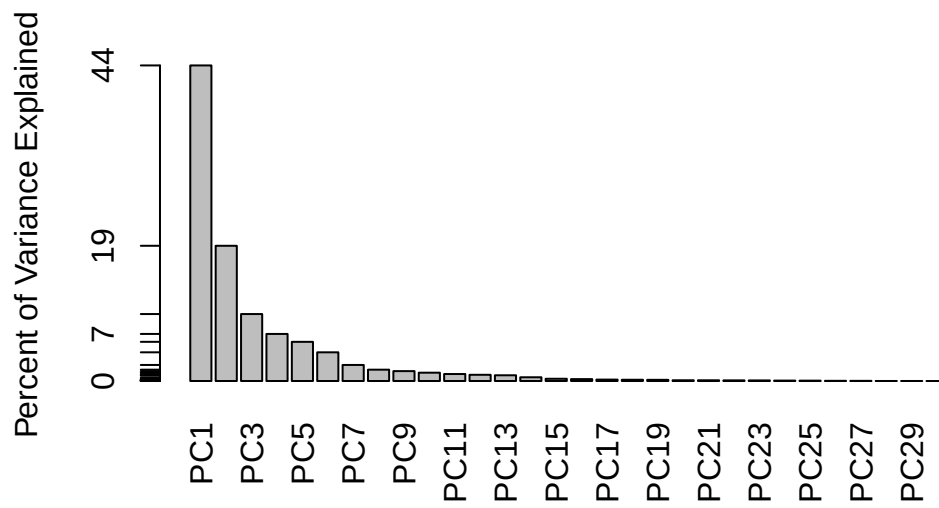
Calculate the variance explained by each principal component by dividing by the total variance explained of all principal components.


```
# Calculate proportion of variance explained (pve)
# Variance explained by each principal component: pve
pve <- pr.var / sum(pr.var)

# Plot variance explained for each principal component
# Scree plot - line plot
plot(c(1,pve), xlab = "Principal Component",
     ylab = "Proportion of Variance Explained",
     ylim = c(0, 1), type = "o")
```



```
# Alternative scree plot - bar plot
barplot(pve, ylab = "Percent of Variance Explained",
        names.arg=paste0("PC",1:length(pve)), las=2, axes = FALSE)
axis(2, at=pve, labels=round(pve,2)*100)
```



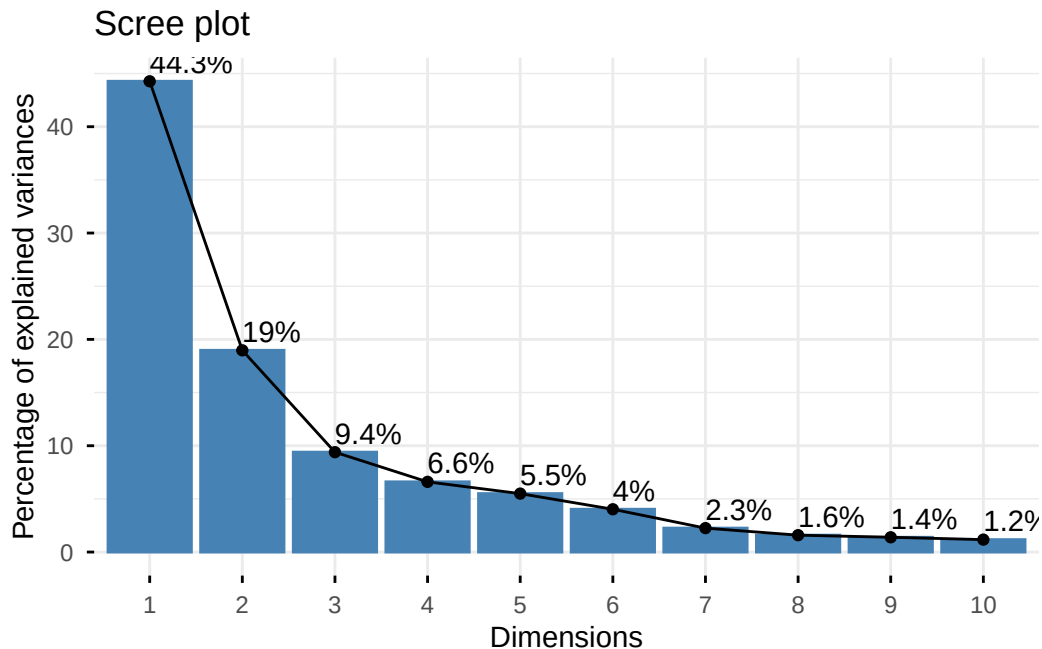
```
## ggplot based graph
#install.packages("factoextra")
library(factoextra)
```

Warning: package 'factoextra' was built under R version 4.5.3

Welcome to factoextra!

Want to learn more? See two factoextra-related books at <https://www.datanovia.com/en/product/guide-to-principal-component-methods-in-r/>

```
fviz_eig(wisc.pr, addlabels = TRUE)
```



Communicating PCA results

- **Q9.** For the first principal component, what is the component of the loading vector (i.e. `wisc.pr$rotation[,1]`) for the feature `concave.points_mean`? This tells us how much this original feature contributes to the first PC. Are there any features with larger contributions than this one?

```
# Q9: What is the loading of concave.points_mean on PC1?
wisc.pr$rotation["concave.points_mean", 1]
```

```
[1] -0.2608538
```

The loading of `concave.points_mean` for PC1 is approximately -0.2609, which is a large negative value. To check if others are larger, sort all PC1 loadings by absolute value:

```
# Sort PC1 loadings by absolute value to find the biggest contributors
sort(abs(wisc.pr$rotation[,1]), decreasing=TRUE)
```

```
concave.points_mean
0.26085376
```

```
concavity_mean
0.25840048
```

```
concave.points_worst
0.25088597
```

compactness_mean	perimeter_worst	concavity_worst
0.23928535	0.23663968	0.22876753
radius_worst	perimeter_mean	area_worst
0.22799663	0.22753729	0.22487053
area_mean	radius_mean	perimeter_se
0.22099499	0.21890244	0.21132592
compactness_worst	radius_se	area_se
0.21009588	0.20597878	0.20286964
concave.points_se	compactness_se	concavity_se
0.18341740	0.17039345	0.15358979
smoothness_mean	symmetry_mean	fractal_dimension_worst
0.14258969	0.13816696	0.13178394
smoothness_worst	symmetry_worst	texture_worst
0.12795256	0.12290456	0.10446933
texture_mean	fractal_dimension_se	fractal_dimension_mean
0.10372458	0.10256832	0.06436335
symmetry_se	texture_se	smoothness_se
0.04249842	0.01742803	0.01453145

concave.points_mean is among the top contributors, but concave.points_worst, concavity_mean, concavity_worst, compactness_mean, perimeter_worst, radius_worst, and perimeter_mean have similar absolute loading. They are all close in magnitude, indicating that many size and shape related features contribute to PC1.

4. Hierarchical Clustering

```
# Scale the wisc.data data using the "scale()" function
data.scaled <- scale(wisc.data)

#Next, calculate the (Euclidean) distances between all pairs of observations in the new scaled data
data.dist <- dist(data.scaled)

#Create a hierarchical clustering model using complete linkage. Manually specify the method and linkage
wisc.hclust <- hclust(data.dist, method="complete", linkage="complete")
```

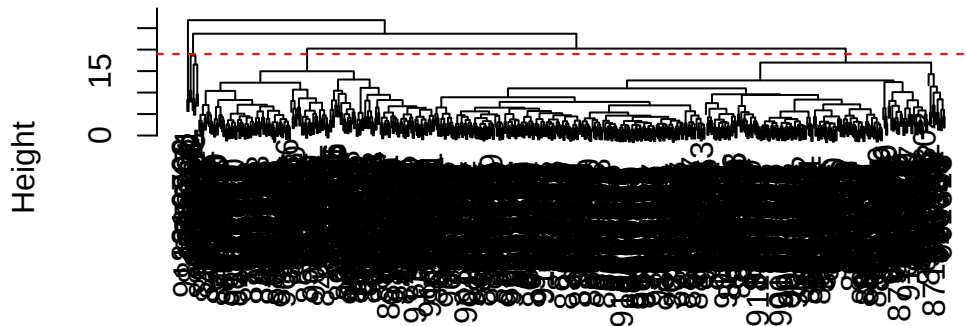
Results of hierarchical clustering

- **Q10.** Using the plot() and abline() functions, what is the height at which the clustering model has 4 clusters?

```
# Plot the dendrogram
plot(wisc.hclust)

# Q10: Add a horizontal line at the height that gives 4 clusters
# By visual inspection of the dendrogram, h 19 gives 4 clusters
abline(h=19, col="red", lty=2)
```

Cluster Dendrogram



```
data.dist
hclust (*, "complete")
```

Selecting number of clusters

```
# Cut the dendrogram into 4 clusters
wisc.hclust.clusters <- cutree(wisc.hclust, k=4)

# Compare clusters to actual diagnoses
table(wisc.hclust.clusters, diagnosis)
```

	diagnosis	
wisc.hclust.clusters	B	M
1	12	165
2	2	5
3	343	40
4	0	2

Q11. OPTIONAL: Can you find a better cluster vs diagnoses match by cutting into a different number of clusters between 2 and 6? How do you judge the quality of your result in each case?

```
table(cutree(wisc.hclust, k=2), diagnosis)
```

	diagnosis	
	B	M
1	357	210
2	0	2

```
table(cutree(wisc.hclust, k=3), diagnosis)
```

	diagnosis	
	B	M
1	355	205
2	2	5
3	0	2

```
table(cutree(wisc.hclust, k=4), diagnosis)
```

	diagnosis	
	B	M
1	12	165
2	2	5
3	343	40
4	0	2

```
table(cutree(wisc.hclust, k=5), diagnosis)
```

	diagnosis	
	B	M
1	12	165
2	0	5
3	343	40
4	2	0
5	0	2

```
table(cutree(wisc.hclust, k=6), diagnosis)
```

	diagnosis	
	B	M
1	12	165
2	0	5
3	331	39
4	2	0
5	12	1
6	0	2

Cutting into $k=2$ gives the worst result, which it puts everything into one giant cluster: 567 samples in cluster 1, so there's essentially no separation. $k=4$ and $k=5$ are similar. $k=5$ just splitting the tiny cluster 2 further but not improving the main separation. $k=6$ starts fragmenting the large benign cluster 3.

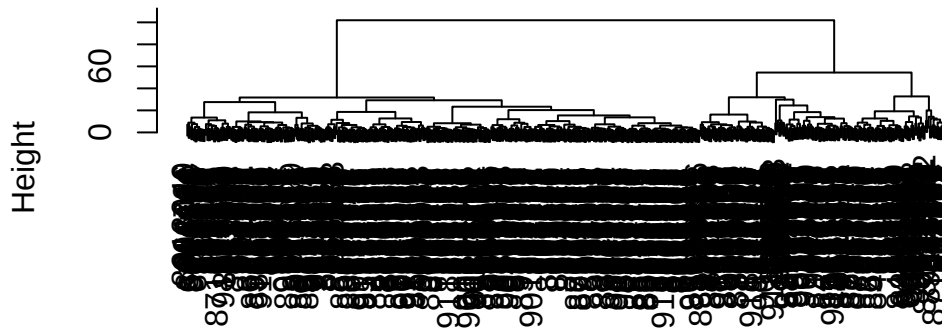
Using different methods

- **Q12.** Which method gives your favorite results for the same `data.dist` dataset? Explain your reasoning.

```
# Try different linkage methods
wisc.hclust.single <- hclust(data.dist, method="single")
wisc.hclust.average <- hclust(data.dist, method="average")
wisc.hclust.ward <- hclust(data.dist, method="ward.D2")

# Plot each and compare
plot(wisc.hclust.ward)
```

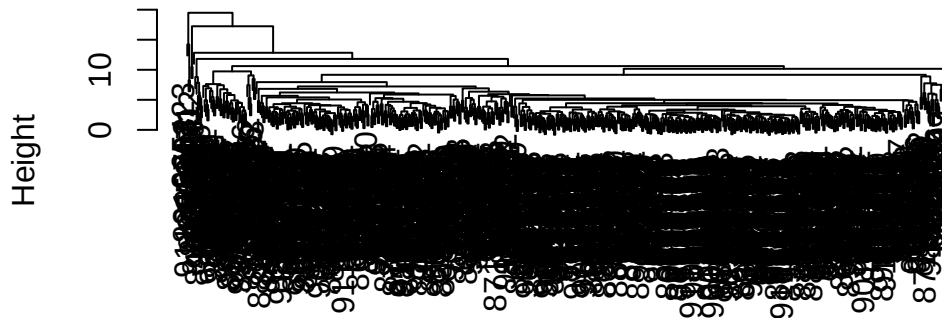
Cluster Dendrogram



```
data.dist  
hclust (*, "ward.D2")
```

```
plot(wisc.hclust.average)
```

Cluster Dendrogram

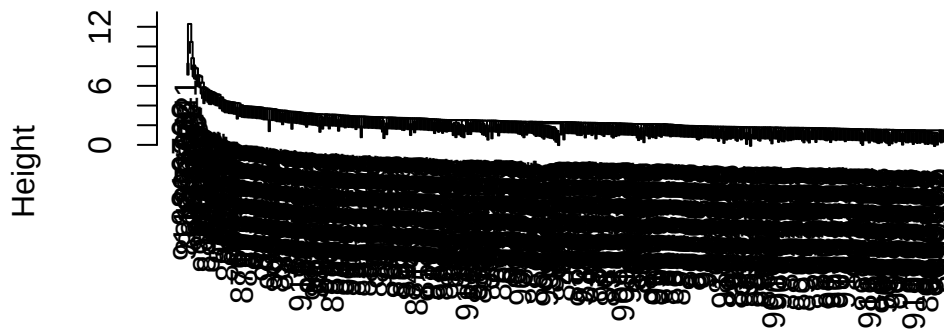


```
data.dist  
hclust (*, "average")
```



```
plot(wisc.hclust.single )
```

Cluster Dendrogram



```
data.dist  
hclust (*, "single")
```

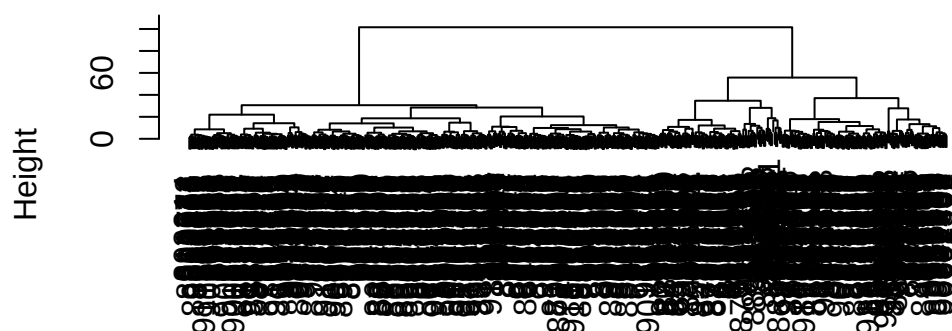
ward.D2 gives the best-looking results. It minimizes within-cluster variance, producing more evenly sized clusters. The dendrogram shows two clear main branches, which aligns well with the expected two-group structure: malignant vs benign. single linkage tends to produce long-chain clusters; complete can create unbalanced groups; average is in between. ward.D2 is best suited for finding spherical, well-separated clusters.

5. Combining methods

Clustering on PCA results

```
# Use the first 7 PCs (covers 90% of variance)  
# Calculate distance matrix on PC scores  
# Then do hierarchical clustering with ward.D2  
wisc.pr.hclust <- hclust(dist(wisc.pr$x[, 1:7]), method="ward.D2")  
  
# Plot show two clear main branches  
plot(wisc.pr.hclust)
```

Cluster Dendrogram



```
dist(wisc.pr$x[, 1:7])
hclust (*, "ward.D2")
```

```
# Cut into 2 groups
grps <- cutree(wisc.pr.hclust, k=2)
table(grps)
```

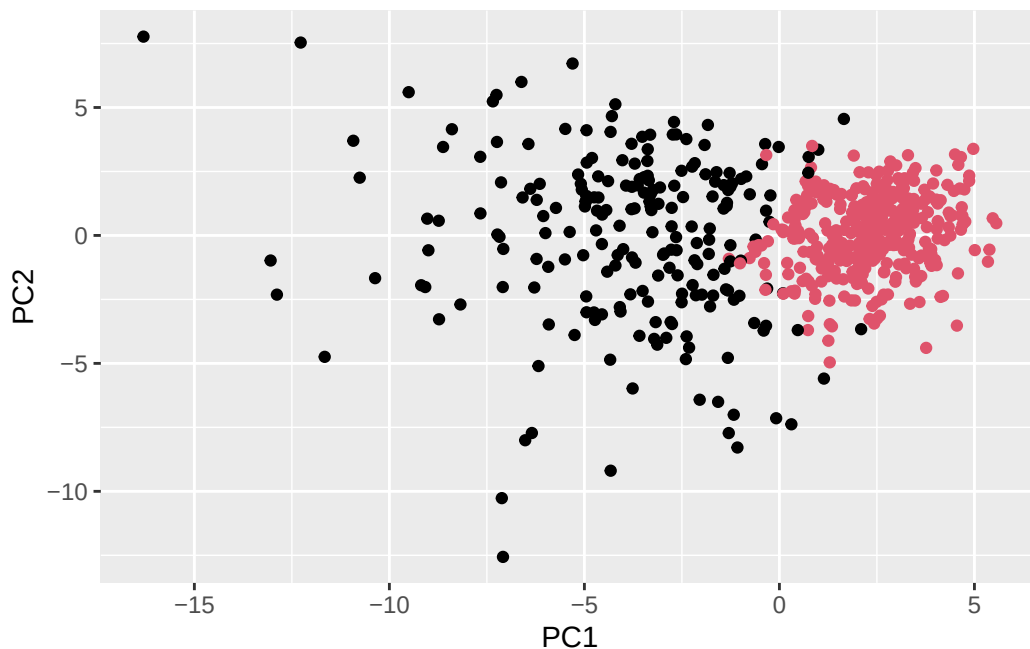
```
grps
  1   2
216 353
```

```
# Compare to actual diagnoses
table(grps, diagnosis)
```

```
      diagnosis
grps    B     M
  1    28  188
  2   329   24
```

Visualize on the PCA score plot

```
ggplot(wisc.pr$x) +
  aes(PC1, PC2) +
  geom_point(col=grps)
```



- **Q13.** How well does the newly created `hclust` model with two clusters separate out the two “M” and “B” diagnoses?

The PCA-based hierarchical clustering with 2 clusters separates M and B well. Cluster 1 contains 188 M and 28 B, which is mostly malignant. Cluster 2 contains 329 B and 24 M, which is mostly benign. Only $28 + 24 = 52$ samples out of 569 are mis-separated, giving an overall accuracy of about 90.9%. This is a strong result for an unsupervised method.

- **Q14.** How well do the hierarchical clustering models you created in the previous sections (i.e. without first doing PCA) do in terms of separating the diagnoses? Again, use the `table()` function to compare the output of each model (`wisc.hclust.clusters` and `wisc.pr.hclust.clusters`) with the vector containing the actual diagnoses.

```
wisc.pr.hclust.clusters <- cutree(wisc.pr.hclust, k=2)
```

```
# Q14: Compare both models side by side
```

```
# PCA + hclust model:
```

```
table(wisc.pr.hclust.clusters, diagnosis)
```

	diagnosis	
wisc.pr.hclust.clusters	B	M
1	28	188
2	329	24

```
# Original hclust model (4 clusters, complete linkage):
table(wisc.hclust.clusters, diagnosis)
```

```

      diagnosis
wisc.hclust.clusters  B  M
1    12 165
2     2  5
3   343 40
4     0  2

```

The hierarchical clustering without PCA (4 clusters, complete linkage) is harder to interpret. It splits the data into 4 groups, with cluster 1 having 12 B mixed into 165 M, and cluster 3 having 40 M mixed into 343 B. The PCA-based model with 2 clusters gives a cleaner 2 split: 28 B are misplaced into the malignant cluster and 24 M are misplaced into the benign cluster. The PCA-based approach is better. It gives a simpler separation because PCA first denoises the data by focusing on the most informative variance directions.

6. Sensitivity/Specificity

- **Q15. OPTIONAL:** Which of your analysis procedures resulted in a clustering model with the best specificity? How about sensitivity?

```
# Sensitivity = TP / (TP + FN)
# TP = malignant samples correctly in the malignant cluster = 188
# FN = malignant samples incorrectly in the benign cluster = 24
sensitivity <- 188 / (188 + 24)
sensitivity
```

```
[1] 0.8867925
```

```
# Specificity = TN / (TN + FP)
# TN = benign samples correctly in the benign cluster = 329
# FP = benign samples incorrectly in the malignant cluster = 28
specificity <- 329 / (329 + 28)
specificity
```

```
[1] 0.9215686
```

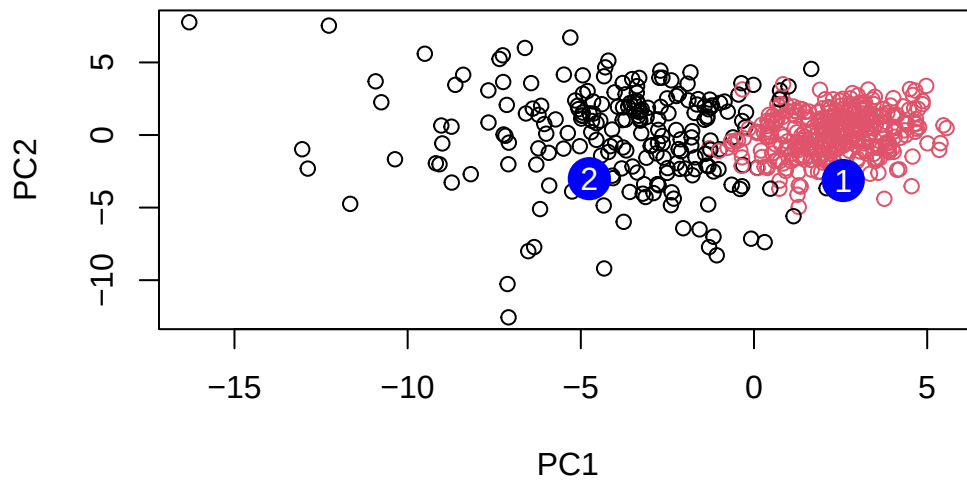
The combined PCA + ward.D2 hclust model gives the best balance of sensitivity 88.7% and specificity 92.2%. The original complete-linkage hclust on raw data has slightly higher sensitivity for malignant detection in cluster 1 ($165/(165+40+5+2) = 77.8\%$ with considering only cluster 1 as M, but the interpretation is harder to understand by having 4 clusters. The PCA-based method provides the cleanest results.

7. Prediction

```
#url <- "new_samples.csv"
url <- "https://tinyurl.com/new-samples-CSV"
new <- read.csv(url)
npc <- predict(wisc.pr, newdata=new)
npc
```

	PC1	PC2	PC3	PC4	PC5	PC6	PC7
[1,]	2.576616	-3.135913	1.3990492	-0.7631950	2.781648	-0.8150185	-0.3959098
[2,]	-4.754928	-3.009033	-0.1660946	-0.6052952	-1.140698	-1.2189945	0.8193031
	PC8	PC9	PC10	PC11	PC12	PC13	PC14
[1,]	-0.2307350	0.1029569	-0.9272861	0.3411457	0.375921	0.1610764	1.187882
[2,]	-0.3307423	0.5281896	-0.4855301	0.7173233	-1.185917	0.5893856	0.303029
	PC15	PC16	PC17	PC18	PC19	PC20	
[1,]	0.3216974	-0.1743616	-0.07875393	-0.11207028	-0.08802955	-0.2495216	
[2,]	0.1299153	0.1448061	-0.40509706	0.06565549	0.25591230	-0.4289500	
	PC21	PC22	PC23	PC24	PC25	PC26	
[1,]	0.1228233	0.09358453	0.08347651	0.1223396	0.02124121	0.078884581	
[2,]	-0.1224776	0.01732146	0.06316631	-0.2338618	-0.20755948	-0.009833238	
	PC27	PC28	PC29	PC30			
[1,]	0.220199544	-0.02946023	-0.015620933	0.005269029			
[2,]	-0.001134152	0.09638361	0.002795349	-0.019015820			

```
# Plot original data colored by cluster assignment,
#overlay new patients as blue points
plot(wisc.pr$x[,1:2], col=grps)
points(npc[,1], npc[,2], col="blue", pch=16, cex=3)
text(npc[,1], npc[,2], c(1,2), col="white")
```



- **Q16.** Which of these new patients should we prioritize for follow up based on your results?

Patient 2 should be prioritized for follow up. In the PCA plot, Patient 2 (PC1 -4.75) falls within the cluster associated with malignant samples, the black/left cluster, while Patient 1 (PC1 +2.58) falls within the benign cluster. Patient 2's projected position among the malignant samples suggests their cell measurements are more consistent with cancerous cells.